

****Timebox: 2 hours.****

What to build

A tiny app to split a bill between people by percentage.

Backend (Go)

- `GET /bills/{id}/shares` — returns the people and their percentages for a bill
- `PUT /bills/{id}/shares` — replaces the full set of shares for that bill
- A schema or migration file for the tables
- ****Rule:**** the percentages for a bill must sum to exactly 100.00. A request that breaks this must be rejected with a clear error.

A bill has an id, a description and a total amount. Seed one bill so the app is usable on first run.

Frontend (Vue 3)

A single page that:

- lists the people on the bill and their percentages
- lets you add and remove people, and edit percentages
- shows the running total of percentages, and each person's amount in euros
- prevents saving when the total isn't 100

Storage

Postgres or SQLite — your choice. Include the schema file.

README

Half a page, no more:

- how to run it (see below)
- decisions you made and why
- what you'd change with a week instead of 2 hours
- what you left out on purpose

Explicitly out of scope

Don't spend time on: authentication, user accounts, visual design, CI, deployment, Dockerfile polish, multiple bills in the UI, or test coverage beyond what's asked below.

Requirements for the submission

1. **It must start with one command** (e.g. ``docker compose up``, ``make run``, or a script). State the command in the README. If it needs more than two commands, we probably won't get it running.
2. **At least one test** proving the sum-to-100 rule rejects a bad payload.
3. A git repo (link or zip) with the history intact — we don't mind messy commits.

On AI tools

Feel free to use them. Copilot, Claude, Cursor, whatever you work with normally — we'd rather see how you actually work.